Thesis Application Proposal

Design and Orchestration of a Cloud-Native Image Processing and AI Insights Pipeline

Nicola Romano

May 2026

Contents

1 Introduction

The project aims to define and implement a cloud-native platform, designed to provide a suite of image processing and analysis services. The platform serves as a practical case study for the "DevOps" course, simulating a professional environment where various technical modules are integrated progressively.

The system will allow users to upload images to a dashboard to perform two main types of tasks:

- **Deterministic Processing:** Leveraging the **FFmpeg** library for image normalization, format conversion (e.g., PNG to JPG), and thumbnail generation.
- **AI-Driven Insights:** Integrating pre-trained models from **Hugging Face** (such as *ViT* for classification or *RMBG* for background removal) to extract metadata and transform assets.

The core of the thesis is not the development of these underlying tools, but rather the **architectural orchestration** required to manage their lifecycle, scaling, and communication within a distributed environment.

2 Motivation, why image processing and AI?

The choice of this application is driven by the specific resource requirements of image processing and AI inference. These tasks are notoriously CPU and RAM-intensive, providing a realistic justification for a **microservices architecture**.
In a monolithic context, heavy processing tasks can easily lead to resource exhaustion, impacting the responsiveness of the entire system. By decoupling these functionalities into independent services, we can explore key DevOps challenges:

- **Heterogeneous Environments:** Coordinating services written in different languages (Java for business logic, Python for AI).
- **Independent Scalability:** Scaling the AI worker nodes horizontally based on demand without affecting the web gateway.
- **Resilience:** Leveraging service mesh traffic management policies (e.g., retries, timeouts, and circuit breakers via Istio) to ensure the core dashboard remains available even when high-latency processing services are under heavy load.

Pedagogical Focus on DevOps Operations Furthermore, this architecture allows the student to maintain a strict focus on DevOps methodologies and platform engineering rather than low-level software development. By delegating complex logic to established libraries (FFmpeg) and pre-trained models (Hugging Face), the "logic" of the application becomes a "black box."

This shift in focus ensures that the student's effort is spent on the primary goals of the course: mastering container orchestration, automating CI/CD pipelines, ensuring observability through a service mesh, and managing the software lifecycle in a cloud-native environment. The project thus serves as a high-fidelity simulation of a modern DevOps role, where the objective is to build a robust "scaffolding" for existing, diverse software components.

3 Roadmap

The project will follow a step-by-step evolution, mirroring the DevOps course modules to demonstrate the transition from legacy structures to modern cloud-native practices.

3.1 Spring Boot-Java Application

3.1.1 Domain Model and Data Layer

The application is built around a relational domain model with three core entities, deliberately designed to exercise the full range of Spring Data JPA features covered in the course:

- **User:** Represents an authenticated user of the platform. Holds profile information and owns a collection of images.
- **Image:** Represents an uploaded asset, linked to its owner via a `@ManyToOne` association to `User`. Each image can be the subject of multiple processing jobs.
- **ProcessingJob:** Represents a single processing request (e.g., a format conversion or an AI classification run) associated with a specific `Image` via a `@ManyToOne` association. Tracks the job type, current status (*PENDING*, *RUNNING*, *DONE*, *FAILED*), and the output result.

This model produces a natural `User` → `@OneToMany` → `Image` → `@OneToMany` → `ProcessingJob` hierarchy, providing concrete use cases for bidirectional associations, `@JoinColumn`, and JPA lazy loading.

Each entity is managed through a dedicated `@Repository` interface extending `JpaRepository`, and all mutating operations are encapsulated in `@Service` classes annotated with `@Transactional` (with appropriate isolation levels), keeping the business logic cleanly separated from the `@RestController` layer.

3.1.2 Monolithic Application

Initial development of a single Spring Boot application managing the UI, file storage, and a simplified version of the processing logic. This phase highlights the limitations of vertical scaling and tight coupling.

The *Image Service* portion of the monolith exposes its resources using **Spring Data REST**, producing a HAL-compliant API discoverable via the HAL Explorer. **Projections** are defined to expose lightweight views of the model (e.g., a projection on `Image` that returns only the filename, status, and creation timestamp without the full `User` or `ProcessingJob` payloads), and **Excerpts** are configured as default collection views. This approach directly exercises the HATEOAS/HAL portion of the course curriculum before the system is decomposed into microservices.

3.1.3 Microservices Decomposition

Refactoring the monolith into two independent Spring Boot services:

- **Gateway Service:** Handles user interaction, serves the dashboard UI, and routes incoming requests to the Image Service via standard `@RestController` endpoints and `RestTemplate` (or `WebClient`) calls.
- **Image Service:** Owns the domain model (`User`, `Image`, `ProcessingJob`) and continues to expose a HAL-compliant Spring Data REST API internally. It coordinates the invocation of the Python AI Worker for heavy processing tasks.

This split creates a clear architectural contrast: the Gateway uses a plain REST controller pattern oriented toward the UI, while the Image Service retains the HATEOAS/Spring Data REST style for its internal resource API. Communication between the two Java services is established via REST, while the Image Service communicates with the Python AI Worker over REST as well.

3.2 Containerization

Packaging the microservices and their dependencies (including FFmpeg and Python AI runtimes) using **Docker**. Creation of a *Docker Compose* environment to orchestrate the multi-language stack (Java, Python, and PostgreSQL) on a local developer machine.

3.3 Deploy on Kubernetes

3.3.1 Single Instance

Transitioning from Docker Compose to a Kubernetes cluster (e.g., Minikube). Defining basic *Deployments*, *Services*, and *Persistent Volumes* for asset storage.

3.3.2 Multiple Instances and Scaling

Implementation of the **Horizontal Pod Autoscaler (HPA)**. The system will be tested to automatically scale the processing nodes in response to an increase in concurrent image upload requests.

3.3.3 Distributed Services with Load Balancer

Configuring an *Ingress Controller* and Load Balancer to manage external access and distribute traffic efficiently across the cluster nodes.

3.4 Testing, CI/CD, Authentication, and Security

3.4.1 Testing

Implementing integration tests using *Testcontainers* to spin up dependent services (e.g., PostgreSQL) as containers during the test phase, ensuring a consistent and reproducible test environment independent of the host machine.

3.4.2 CI/CD

Developing an automated pipeline using **GitLab CI/CD** (configured via a `.gitlab-ci.yml` file) for building, testing, and pushing images to the **GitLab Container Registry**. The pipeline will include three stages:

- **Test:** Running tests
- **Build:** Building and pushing a Docker image to the GitLab Container Registry.
- **Deploy:** Implementing continuous delivery to a staging environment and continuous deployment to a production environment on the cloud cluster.

3.4.3 Security

Integrating **Keycloak** as the identity provider, deployed as a Docker container and configured with a dedicated realm and client for the application. The Spring Boot services will be secured via **Spring Security**, accepting only requests carrying a valid **JWT** access token issued by Keycloak (using the OAuth 2.0 Authorization Code flow). This ensures only authenticated users can trigger resource-heavy AI tasks.

3.5 Service Mesh

Integration of **Istio** to manage the service-to-service communication. Resilience patterns such as circuit breakers, retries, and timeouts will be implemented at the **mesh level** through Istio's traffic management policies (VirtualService and DestinationRule resources), rather than in application code, keeping the services themselves decoupled from these concerns. This final phase focuses on:

- **Observability:** Using **Kiali** (service mesh topology and circuit breaker visualization), **Jaeger** (distributed tracing), **Prometheus** (metrics collection), and **Grafana** (metrics dashboarding) to visualize traffic flows and identify bottlenecks.
- **Traffic Management:** Implementing advanced policies like *retries*, *timeouts*, and *canary deployments* for new AI model versions.
- **Security:** Enforcing mutual TLS (mTLS) for service-to-service communication and JWT-based request authentication at the mesh level.

3.6 Cloud Deployment and Configuration Management

3.6.1 Cloud Infrastructure Provisioning

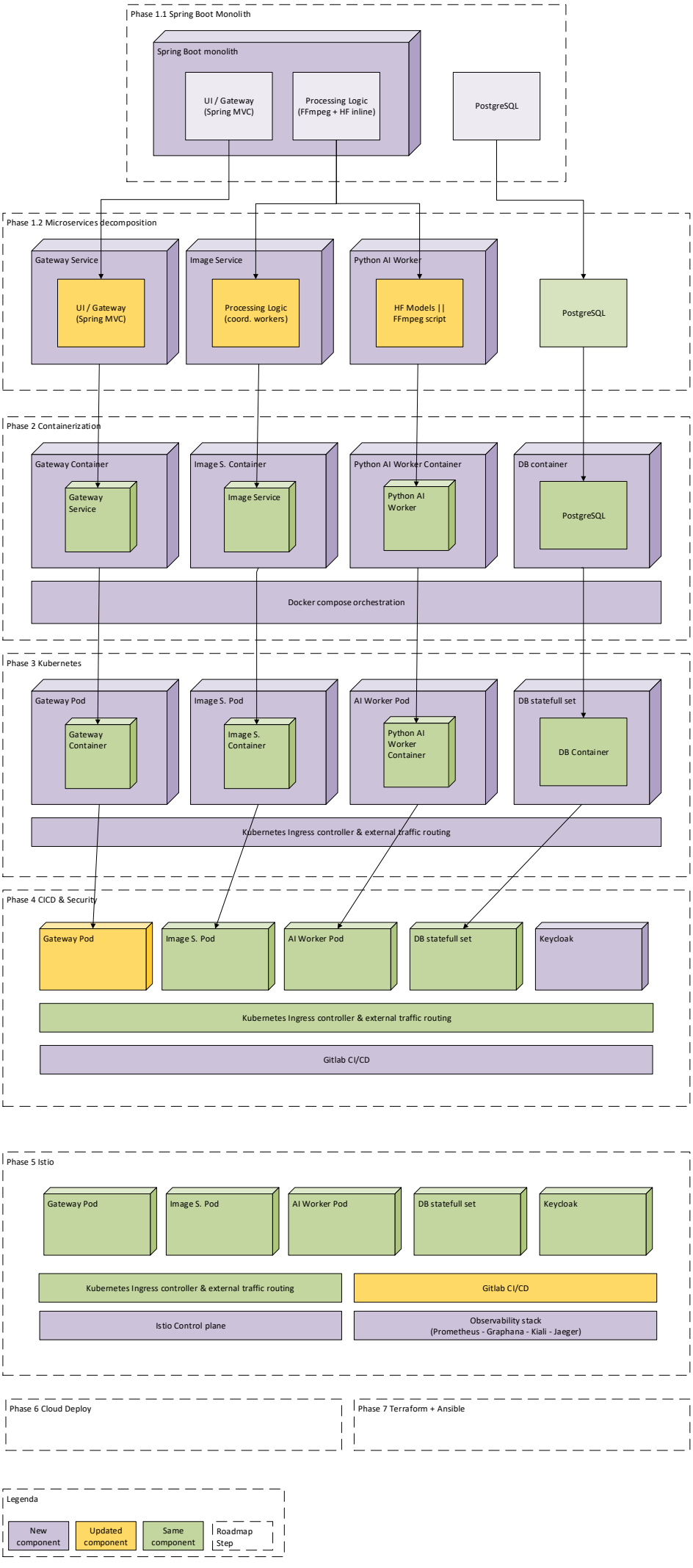
Transitioning the cluster from a local environment to **GCP** using GKE, **Azure** using AKS or **AWS** using EKS. This phase includes managing cloud-specific constraints such as disk quotas and regional availability.

3.6.2 Configuration Management with Helm

Using **Helm** to package and manage the application deployment as a reusable Chart. By templating the Kubernetes manifests, a single Chart can be deployed consistently across the *dev*, *staging*, and *production* environments simply by providing different `values.yaml` configuration files. The GitLab CI/CD pipeline will drive these deployments automatically.

3.7 Bonus: Infrastructure as Code (IaC)

As an optional extension beyond the core course scope, **Terraform** may be used to define the cloud infrastructure as code. This would include the automated provisioning of the cloud cluster, VPC networking, and cloud storage buckets (GCS) required for image persistence. Additionally, **Ansible** could be explored for configuration management tasks that complement Terraform's infrastructure provisioning. This phase is considered a stretch goal and will be pursued if time permits after the core roadmap is complete.


Phase 1.1 Spring Boot Monolith
Spring Boot monolith
UI / Gateway (Spring MVC)
Processing Logic (FFmpeg + HF inline)
PostgreSQL
Phase 1.2 Microservices decomposition
Gateway Service
UI / Gateway (Spring MVC)
Image Service
Processing Logic (coord. workers)
Python AI Worker
HF Models || FFmpeg script
PostgreSQL
Phase 2 Containerization
Gateway Container
Gateway Service
Image S. Container
Image Service
Python AI Worker Container
Python AI Worker
DB container
PostgreSQL
Docker compose orchestration
Phase 3 Kubernetes
Gateway Pod
Gateway Container
Image S. Pod
Image S. Container
AI Worker Pod
Python AI Worker Container
DB statefull set
DB Container
Kubernetes Ingress controller & external traffic routing
Phase 4 CICD & Security
Gateway Pod
Image S. Pod
AI Worker Pod
DB statefull set
Keycloak
Kubernetes Ingress controller & external traffic routing
Gitlab CI/CD
Phase 5 Istio
Gateway Pod
Image S. Pod
AI Worker Pod
DB statefull set
Keycloak
Kubernetes Ingress controller & external traffic routing
Gitlab CI/CD
Istio Control plane
Observability stack (Prometheus - Graphana - Kiali - Jaeger)
Phase 6 Cloud Deploy
Phase 7 Terraform + Ansible
Legenda
New component
Updated component
Same component
Roadmap Step